

Sistema de Predicción de Enfermedades con Redes Neuronales Recurrentes

San Luis Potosí, México — Hantavirus, COVID-19 y Dengue

José Francisco Hernández Martínez

IICO — Universidad Autónoma de San Luis Potosí

24 de mayo de 2026



Contenido

1. Introducción
2. Fundamentos Teóricos
3. Metodología
4. Resultados
5. Stack Tecnológico

- Las enfermedades infecciosas representan un desafío constante para la salud pública
- San Luis Potosí presenta incidencia de hantavirus, COVID-19 y dengue
- Las herramientas predictivas permiten anticipar brotes y optimizar recursos
- Integración de datos epidemiológicos + tendencias de búsqueda web (Google Trends)

Objetivo

Construir un sistema de predicción temprana basado en RNN que integre datos epidemiológicos y tendencias de búsqueda para 58 municipios de SLP.

Arquitectura del Sistema

- 1 Recolección: Google Trends + APIs gubernamentales + CSV manual
- 2 Almacenamiento: SQLite con 5 tablas normalizadas
- 3 Modelo: LSTM bidireccional (2 capas, 64–128 unidades)
- 4 Visualización: Dashboard web con Plotly.js

Redes Neuronales Recurrentes (RNN)

- Las RNN capturan dependencias temporales en secuencias
- A diferencia de las CNN, mantienen un estado oculto que evoluciona con el tiempo

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

- **Problema:** gradientes explosivos/desvanecientes en secuencias largas
- **Solución:** LSTM (Long Short-Term Memory) con compuertas de olvido, entrada y salida

LSTM — Long Short-Term Memory

Compuertas:

- Olvido: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- Entrada:
 $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- Salida: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

Estado de celda:

- $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$
 - $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$
 - $h_t = o_t * \tanh(C_t)$
-
- La capa bidireccional procesa la secuencia en ambas direcciones
 - Captura patrones antes y después de cada punto temporal

Arquitectura del Modelo

Componente	Detalle
Entrada	21 días $\times$ N características
Capa LSTM Bidireccional 1	64 unidades, return sequences
Dropout	0.3
Capa LSTM Bidireccional 2	128 unidades
Dropout	0.3
Salida 1 (Clasificación)	Sigmoide $\rightarrow$ probabilidad de brote
Salida 2 (Regresión)	ReLU $\rightarrow$ casos estimados en 7 días

- Pérdida combinada: `binary_crossentropy` + `mse`
- Optimizador: Adam, learning rate 10^{-3}
- Early stopping con paciencia de 10 épocas

- ❶ **Google Trends:** Extracción por palabra clave por municipio (vía pytrends)
 - 5–7 palabras clave por enfermedad
 - Fallback graceful ante HTTP 429
- ❷ **Datos epidemiológicos:** Fuentes oficiales (Datos Abiertos México, SINAIS)
 - Casos confirmados, hospitalizaciones, muertes
 - Stubs para desarrollo sin conexión real
- ❸ **Importación CSV:** Alternativa manual con validación de formato
- ❹ **Datos sintéticos:** Generación con patrones estacionales para 58 municipios

Características por Enfermedad

Cada enfermedad tiene un conjunto específico de palabras clave de Google Trends:

Enfermedad	Keywords	Características
Dengue	8	$8 + 3 = 11$
Hantavirus	10	$10 + 3 = 13$
COVID-19	7	$7 + 3 = 10$

Las 3 características epidemiológicas comunes:

- Casos confirmados (normalizados $\div 100$)
- Hospitalizaciones (normalizadas $\div 20$)
- Muertes (normalizadas $\div 5$)

Generación de Datos Sintéticos

- Período: 2 años (730 días)
- 58 municipios con pesos poblacionales
- Patrones estacionales realistas:
 - **Dengue**: pico en temporada de lluvias (junio–octubre)
 - **COVID-19**: pico en invierno (diciembre–febrero)
 - **Hantavirus**: brotes en primavera y otoño
- Ruido aleatorio para simular variabilidad real
- 29,750 registros de tendencias + 152,250 registros epidemiológicos

Modelo	Sensibilidad	Precisión	RMSE
Dengue	1.000	1.000	581.58
Hantavirus	1.000	1.000	484.33
COVID-19	1.000	1.000	741.35

- Sensibilidad perfecta en datos sintéticos por etiquetado agregado
- RMSE refleja magnitud de casos; COVID-19 presenta mayor variabilidad
- En producción con datos reales se espera sensibilidad $> 0,75$

- FastAPI como backend (8 endpoints REST)
- Plotly.js para visualizaciones interactivas
- Actualización automática cada 60 segundos
- Tres vistas: dengue, hantavirus, COVID-19

Tarjeta de Predicción

- Probabilidad de brote (0–100 %)
- Casos estimados a 7 días
- Intervalo de confianza al 95 %

Métricas del Modelo

- Sensibilidad, especificidad
- RMSE, precisión

Serie Temporal

- Casos vs hospitalizaciones vs muertes
- Ventana configurable (7–365 días)

Mapa de Calor

- Distribución por municipio
- 58 municipios de SLP
- Código de colores por intensidad

Componente	Tecnología
Lenguaje	Python 3.11
Deep Learning	TensorFlow 2.18 / Keras
Backend	FastAPI 0.115 (Uvicorn)
Base de datos	SQLite (psycopg2 para PostgreSQL opcional)
Frontend	HTML5 + Plotly.js 2.32
Tendencias	pytrends (Google Trends)
Testing	pytest 8.3
Estilo	Código limpio sin comentarios

Estructura del Proyecto

src/app.py — FastAPI
src/database.py — SQLite CRUD
src/predictor.py — Inferencia RNN
src/train_model.py — Entrenamiento
src/synthetic_data.py — Datos
sinteticos
src/import_csv.py — Importacion CSV
src/update_from_apis.py — Pipeline
diario
src/data_sources/google_trends.py
src/data_sources/government_apis.py
tests/ — 5 módulos de prueba
static/ — HTML + CSS + JS
models/ — Modelos .keras
data/database.db — SQLite poblada

32 tests unitarios
8 endpoints REST
3 modelos entrenados
4 scripts independientes

- Sistema funcional de predicción de enfermedades con RNN para SLP
- Integración exitosa de múltiples fuentes de datos
- Dashboard web interactivo con actualización en tiempo real
- Arquitectura modular permite agregar nuevas enfermedades fácilmente
- Diseñado para funcionar sin conexión a internet (datos sintéticos + CSV)

- **Fase 2:** Integración con Google Forms para captura de datos manual
- **Validación clínica:** Evaluación del modelo con datos reales de SLP
- **Modelos avanzados:** Experimentar con Transformers y atención temporal
- **Escalabilidad:** Migrar a PostgreSQL y desplegar en servidor Cloud
- **Aplicación móvil:** Notificaciones push para alertas de brotes

¿Preguntas?

Código fuente disponible en GitHub

-  S. Hochreiter y J. Schmidhuber, *Long Short-Term Memory*, Neural Computation, 1997.
-  F. Chollet, *Deep Learning with Python*, Manning, 2018.
-  Google LLC, *Google Trends API*, trends.google.com
-  S. Ramírez, *FastAPI*, fastapi.tiangolo.com